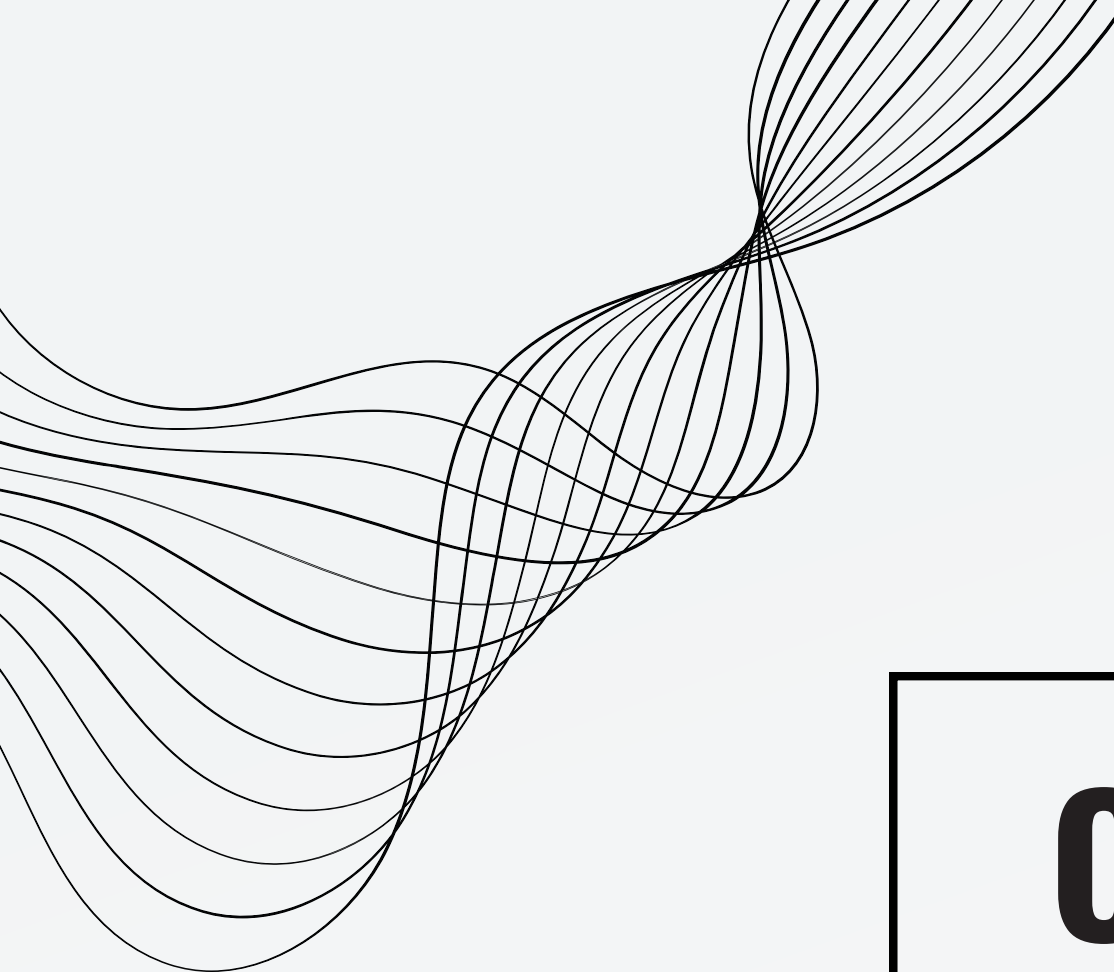
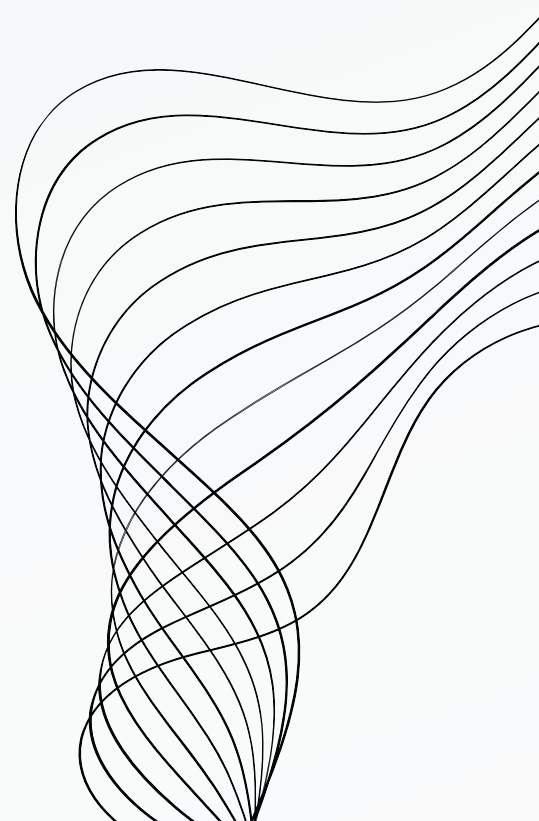




CLEANINGROBOTPY PROJECT

(FOR THE BEST EMBEDDED SYSTEMS COURSE EVER)

ZAKARYA BOUDRAF



User Story #6 UV Light

- **WHEN THE ROBOT MOVES (USER STORY #3), A UV LIGHT ATTACHED TO THE BOTTOM OF THE ROBOT IS ACTIVATED FOR 30 SECONDS TO DISINFECT THE AREA.**
- **THE UV LIGHT SHARES THE SAME FEATURES AS THE RECHARGING LED AND IS CONNECTED TO PIN 7 (BOARD MODE).**
- **THE COMMUNICATION WITH THE UV LIGHT HAPPENS VIA THE GPIO.OUTPUT(CHANNEL, VALUE) FUNCTION, WHERE CHANNEL IS THE PIN NUMBER AND VALUE IS FALSE | TRUE.**
- **THE UV LIGHT HAS ALREADY BEEN CONFIGURED IN THE CONSTRUCTOR OF THE CLEANINGROBOT CLASS.**

Red Phase #1

```
@patch.object(GPIO, attribute: "output")  👤 Zakarya
@patch("time.sleep")
def test_activate_uv_light(self, mock_sleep: Mock, mock_uv_light: Mock):
    system = CleaningRobot()

    system.activate_uv_light()

    mock_uv_light.assert_any_call(*args: system.UV_LIGHT_PIN, True)

    mock_uv_light.assert_any_call(*args: system.UV_LIGHT_PIN, False)

    self.assertTrue(mock_sleep.called)
    mock_sleep.assert_called_with(30)
```

Green Phase #1

```
def activate_uv_light(self) -> None: 2 usages  👤 Zakarya  
    GPIO.output(self.UV_LIGHT_PIN, value: True)  
    time.sleep(30)  
    GPIO.output(self.UV_LIGHT_PIN, value: False)
```

Red Phase #2

```
@patch.object(IBS, attribute: "get_charge_left")  ⤵ Zakarya
@patch.object(CleaningRobot, attribute: "activate_uv_light")
def test_uv_light_activation_on_movement(self, mock_activate_uv_light: Mock, mock_ibs: Mock):
    mock_ibs.return_value = 11
    system = CleaningRobot()
    system.pos_x = 1
    system.pos_y = 1
    system.heading = "N"

    system.execute_command("f")

    mock_activate_uv_light.assert_called_once()

    self.assertEqual(system.pos_x, second: 1)
    self.assertEqual(system.pos_y, second: 2)
    self.assertEqual(system.heading, second: "N")
```


Green Phase #2

```
if command == "f":
    if self.obstacle_found():
        if self.heading == "N":
            obstacle_pos = (self.pos_x, self.pos_y + 1)
        elif self.heading == "S":
            obstacle_pos = (self.pos_x, self.pos_y - 1)
        elif self.heading == "E":
            obstacle_pos = (self.pos_x + 1, self.pos_y)
        elif self.heading == "W":
            obstacle_pos = (self.pos_x - 1, self.pos_y)
        return f"{self.robot_status()}({obstacle_pos[0]},{obstacle_pos[1]})"
    if self.heading == "N":
        self.pos_y += 1
    elif self.heading == "S":
        self.pos_y -= 1
    elif self.heading == "E":
        self.pos_x += 1
    elif self.heading == "W":
        self.pos_x -= 1
    self.activate_wheel_motor()
    self.activate_uv_light()
    return self.robot_status()
```